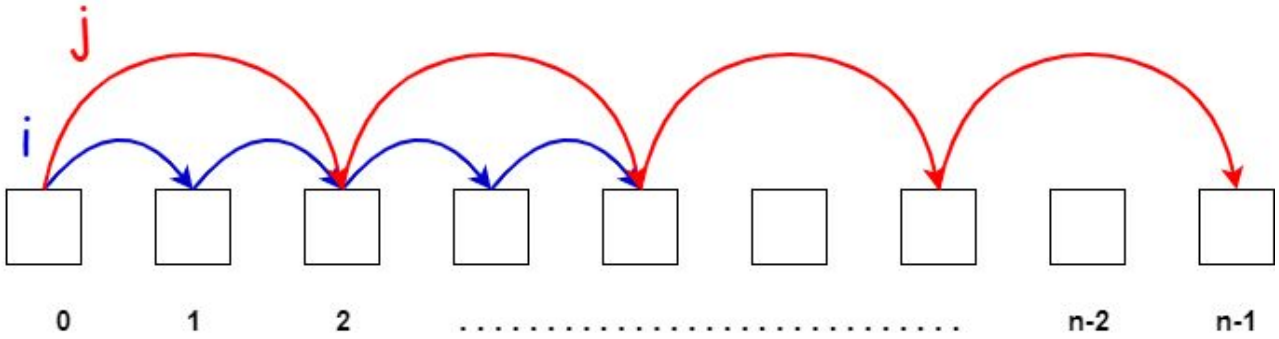
Two Pointers



Lecture Flow

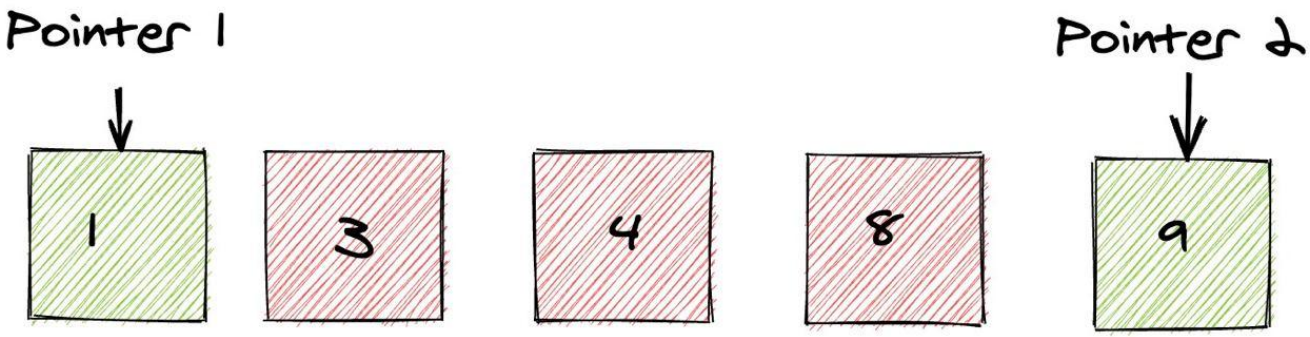
1) Pre-requisites
2) Definitions
3) Different Variants
4) Things to Pay Attention (common pitfalls)
5) Practice Questions
6) Resources
7) Quote of the Day

Pre-requisites

- Basic flow control (for, while loops)
- Linear data structures

Definitions

Two Pointers technique is the use of two different pointers (usually to keep track of array or string indices) to solve a problem involving said indices with the benefit of saving time and space.



Different Variants

A2SV
Africa To Silicon Valley

Variants

Variant I: Parallel Pointers

Variant II: Pointers on Separate Arrays

Variant III: Colliding Pointers

Variant IV: Seeker and Placeholder

Variant V: For-While Combo

Variant I: Parallel Pointers

A2SV
Africa To Silicon Valley

Problem Pattern

Given an array of integers, determine if it is sorted non-decreasing order.

Input

An array of integers.

Output

True or false, whether or not the array is sorted.

Approach Pattern

In this problem, we only need to look at two consecutive values. This is because for three consecutive numbers a, b, and c, if a <= b and b <= c, then a <= c.

The two pointers will iterate parallel to each other, until the right-most one reaches the end of the array.

Practice

Problem Link

Implementation

```
def isSorted(nums):
    left = 0
    right = 1
    size = len(nums)
    while right < size:
        if nums[left] > nums[right]:
            return False
        left += 1
        right += 1

    return True
```

Variant II: Pointers on Separate Arrays

A2SV
Africa To Silicon Valley

Problem Pattern

You are given two arrays, sorted in non-decreasing order. Merge them into one sorted array.

Input

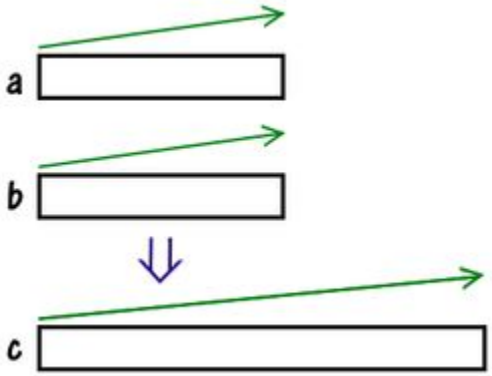
Two sorted arrays of integers.

Output

The merged array.

Approach Pattern

What is this task? We are given two arrays, a and b, sorted in ascending order. We want to combine the elements of these arrays into one big array c, also sorted in ascending order.



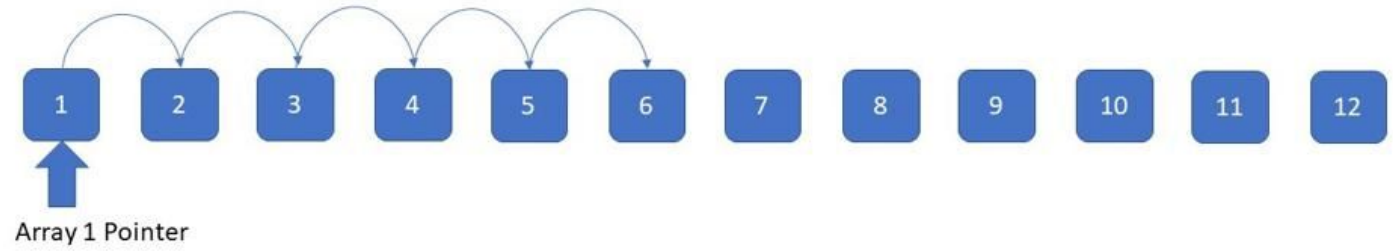
The easiest way to do this is with the following algorithm:

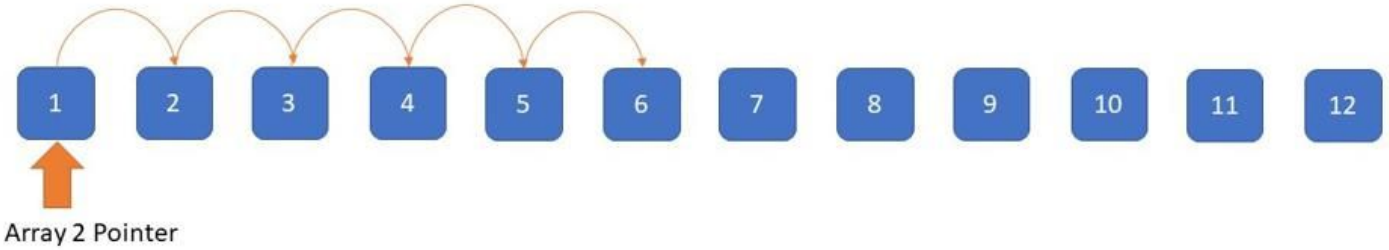
1. Collect all the elements into one big array;
2. Sort it with any sorting method built into your language.

Such an algorithm will take time $O((m+n) \cdot \log(m+n))$

Efficient Approach: Pointer on Separate Arrays

- Two arrays or Lists, each has been assigned a pointer



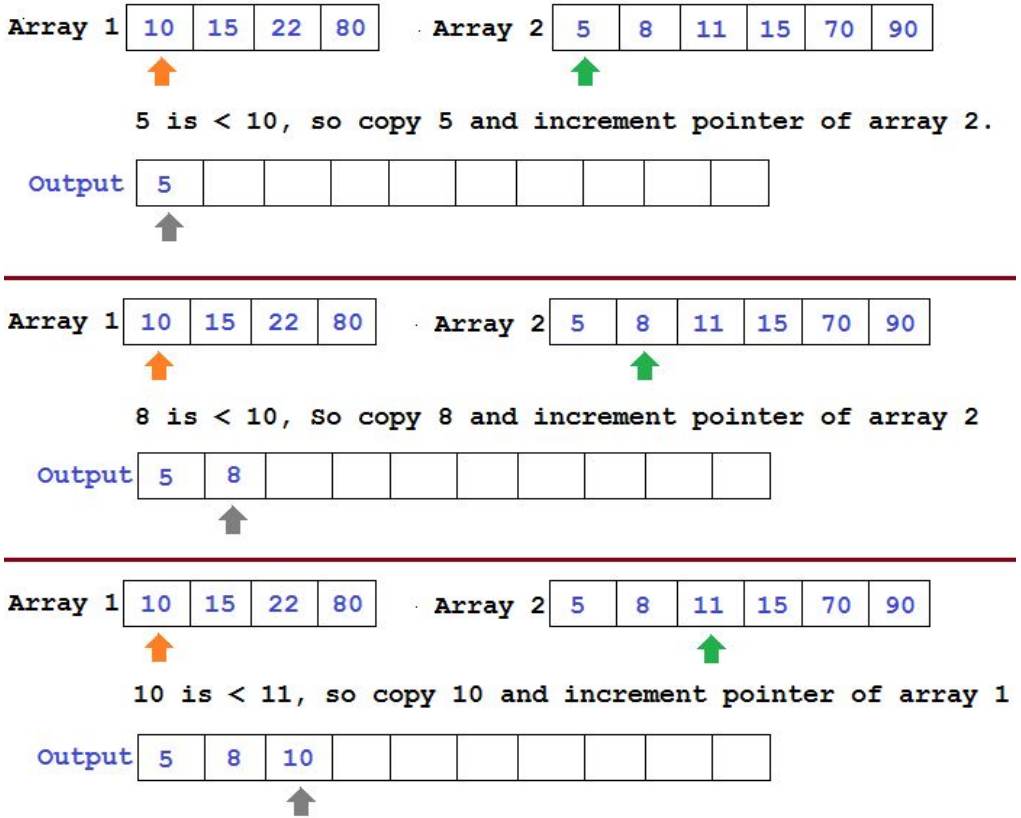


Simulation

How do we merge them into one sorted array?

To answer this question, let's understand which element will be in the first position in this array. Of course, this is the smallest element among all in a and b. The smallest element of a is at the beginning of the array, same as with b.

Simulation


Array 1
10
15
22
80
Array 2
5
8
11
15
70
90
5 is < 10, so copy 5 and increment pointer of array 2.
Output
5
Array 1
10
15
22
80
Array 2
5
8
11
15
70
90
8 is < 10, So copy 8 and increment pointer of array 2
Output
5
8
Array 1
10
15
22
80
Array 2
5
8
11
15
70
90
10 is < 11, so copy 10 and increment pointer of array 1
Output
5
8
10

Practice

Problem Link

Implementation

```
def mergeLists(list1, list2):
    merged = []
    first, second = 0, 0

    while first < len(list1) and second < len(list2):

        if list1[first] < list2[second]:
            merged.append(list1[first])
            first += 1

        else:
            merged.append(list2[second])
            second += 1

    merged.extend(list1[first:])
    merged.extend(list2[second:])


    return merged
```

Variant III: Colliding Pointers

A2SV
Africa To Silicon Valley

Problem Pattern

Given an array of integers that is sorted in non-decreasing order, find two numbers such that they add up to a specific target number (it is guaranteed that at least one pair exists).

Input

A sorted array of integers and a target number.

Output

Two numbers from the array that sum up to the target number.

Brute Force Approach

- We could implement a nested loop finding all possible pairs of elements and adding them.

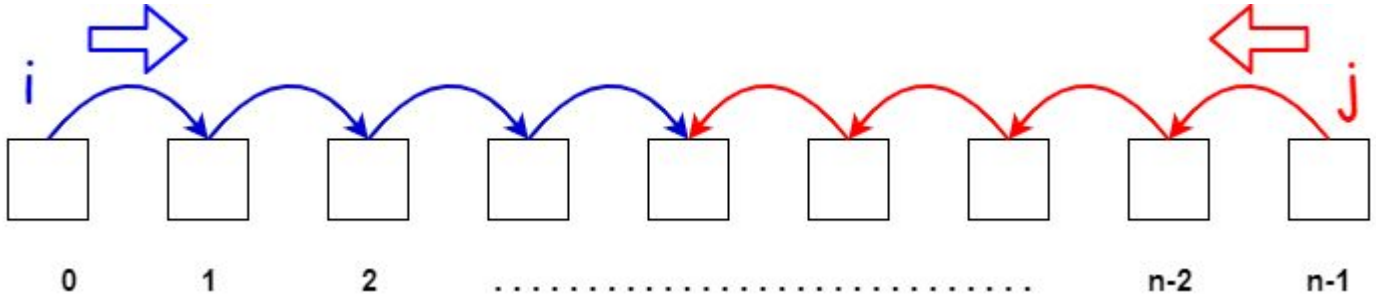
```
for i in range(n-1):
    for j in range(i+1):
        if arr[i] + arr[j] == target:
            return [arr[i], arr[j]]

return []
```

- *Time complexity: $O(n^2)$*

Efficient Approach

- One pointer starts from beginning and other from the end and they proceed towards each other.



Approach

Since the array is sorted, we can make some general observations:

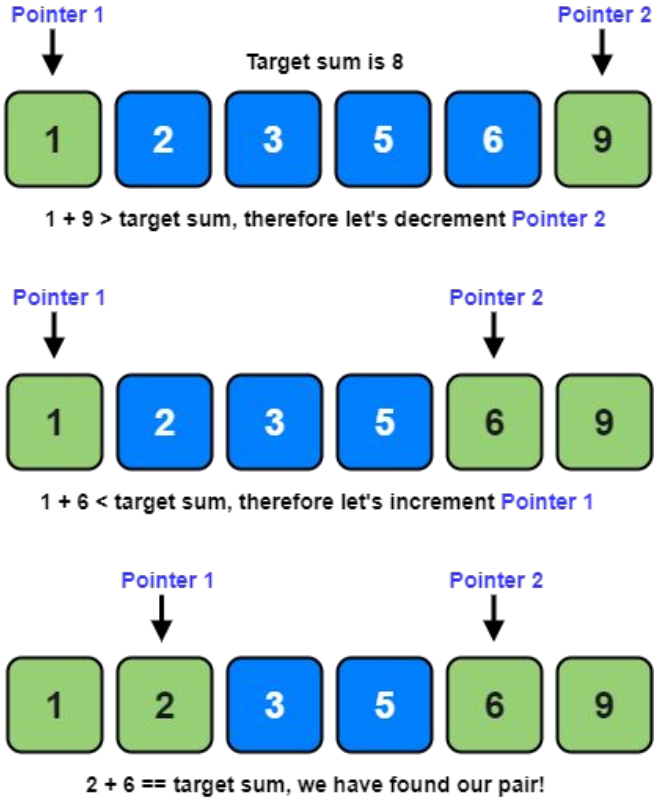
- Smaller sums would come from the left half of the array
- Larger sums would come from the right half of the array

Therefore, using **two pointers** starting at the end points of the array, we can choose to increase or decrease our current sum however we like.

The basic idea is that:

- If our current sum is too small, move closer to the right.
- If our current sum is too large, move closer to the left.

Simulation


Pointer 1
Pointer 2
Target sum is 8
1
2
3
5
6
9
1 + 9 > target sum, therefore let's decrement Pointer 2
Pointer 1
Pointer 2
1
2
3
5
6
9
1 + 6 < target sum, therefore let's increment Pointer 1
Pointer 1
Pointer 2
1
2
3
5
6
9
2 + 6 == target sum, we have found our pair!

Practice

Problem Link

Implementation

```
def twoSum(nums, target):
    left = 0
    right = len(nums) - 1
    cur_sum = 0
    while nums[left] + nums[right] != target:
        cur_sum = nums[left] + nums[right]
        if cur_sum < target:
            left += 1
        else:
            right -= 1

    return [left+1, right+1]
```

Variant IV: Seeker and Placeholder

A2SV
Africa To Silicon Valley

Approach Pattern

This approach is used when we are trying to group elements based on some criteria and the array has to be modified in place.

One pointer is used to seek valid elements for the group, and the other is used to keep track of the next valid position for an element.

Problem

You are given an array, group all non-zero elements to the beginning of the array while maintaining their **relative order**. The modification must be done **in-place**.

Input

An array of integers.

Output

The array, modified in-place, such that all non-zero numbers are at the beginning of the array.

Approach

Any approaches to solve this problem?

Approach

- One pointer will iterate over the array, finding non-zero elements. The other pointer will point to the next valid position for a non-zero element.

Simulation

[0,4,0,1,3]

Simulation

[0,4,0,1,3]

Holder
Seeker

Simulation

[0,4,0,1,3]

Holder
Seeker

Simulation

[4,0,0,1,3]

Holder
Seeker

Simulation

[4,0,0,1,3]

Holder
Seeker

Simulation

[4,0,0,1,3]

Holder
Seeker

Simulation

[4,1,0,0,3]

Holder
Seeker

Simulation

[4,1,0,0,3]

Holder
Seeker

Simulation

[4,1,3,0,0]

Holder
Seeker

Simulation

[4,1,3,0,0]

Holder
Seeker

Practice

Problem Link

Implementation

```
def moveNonZeroes(nums):
    holder = 0
    seeker = 0

    while seeker < len(nums):
        if nums[seeker] != 0:
            nums[seeker], nums[holder] = nums[holder], nums[seeker]
            holder += 1
        seeker += 1
```

Variant V: For-While Combo

A2SV
Africa To Silicon Valley

Approach Pattern

- The for-while combination is used when one pointer moves one step at a time, but the other one moves multiple steps at a time.
- This is usually applied to problems in which the position of one pointer is directly dependent on the position of the other.

Problem

You are given two arrays, sorted in non-decreasing order. For each element of the second array, find the number of elements in the first array that are strictly less than it.

Input

Two sorted arrays.

Output

The number of elements of the first array less than each of the elements of the second array

Approach

Any approaches to solve this problem?

Approach

- What would a brute force approach look like?
- For every element in the second array, iterate over the first array and count the ones that are smaller than it.
- This is inefficient - $O(n^2)$

Approach

- Remember that both arrays are sorted.
- This means that if there are n elements that are less than a specific target, then there is at least n guaranteed elements for the next target too.

Simulation

- We are going to move pointer **j** over array **b** and move **i** over array **a** until we find an a_i from array **a** that is greater than or equal to b_j. At this point, the value of **i** will represent the number of elements in **a** that are less than b_j.

```
a = [2,2,5,6]
b = [3,4,6,8]
c = []
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = []
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = []
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = [2]
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = [2,2]
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = [2,2]
```

Simulation

```
a = [2,2,5,6]

b = [3,4,6,8]

c = [2,2,3]
```

Simulation

```
a = [2,2,5,6]
b = [3,4,6,8]
c = [2,2,3]
```

Simulation

```
a = [2,2,5,6]
b = [3,4,6,8]
c = [2,2,3,4]
```

Practice

Problem Link

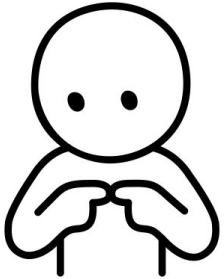
Implementation

```
def countSmaller(nums1, nums2):
    smaller_counts = []
    first = 0

    for second in range(len(nums2)):
        while first < len(nums1) and nums1[first] < nums2[second]:
            first += 1

        smaller_counts.append(first)

    return smaller_counts
```



Common Pitfalls

A2SV
Africa To Silicon Valley

1. Index out of Bounds

Trying to access elements using indices that are **greater (or equal to)** than the size of our array will result in this exception.

Testcase **Result**

```
IndexError: list index out of range
    print(nums[i])
Line 4 in twoSum (Solution.py)
    ret = Solution().twoSum(param_1, param_2)
Line 28 in _driver (Solution.py)
    _driver()
Line 39 in <module> (Solution.py)
```

Stdout

```
2
```

Console ^ Run Submit

A2SV
Africa To Silicon Valley

2. Look out for pointer Condition

One common mistake we make while doing two pointer problems is not paying attention to our guard condition.

```
# left -> our left pointer
# right -> our right pointer
# what should our condition be? What else is wrong with it

while left < right:
    pass

while left <= right:
    pass
```

A2SV
Africa To Silicon Valley

Checkpoint

Link

A2SV
Africa To Silicon Valley

Practice Problems

Divide Players into Teams of Equal Skill

Boats to Save People

Container With Most Water

Sum of Square Numbers

Partition Labels

Merge Sorted Array

Remove Duplicates from Sorted Array

Rotate Array

Alternating Subsequence

Helpful Resources

Codeforces Pilot Course

Geeksforgeeks

Algodaily

There are no foolish questions, and no man becomes a fool until he has stopped asking questions.

(Charles Proteus Steinmetz)

Creator 2023

Tamirat Dereje
Hailemariam Arega
Nazrawi Demeke

Henok Matheas Kebede
Bruk Mekonnen
Misganaw Berihun

Wubshet Zeleke
Yeabsira Driba
Ibsa Abraham

izquotes.com